PID GUI Ideas (Student Lab Focus)

IMPORTANT

Wanted data structure in END

```
timestamp, run_id, heatsink_id, sp_temp, temp, amb_temp, pwm_heater, fan_cmd,
fan_pwm, airspeed, delta_p1, delta_p2, vin, iin, pin, mode, state, event
```

Prioritization Framework

Use this scoring model for each backlog item:

- `Importance` (1..5): impact on heatsink-comparison quality.
- `Necessity` (1..5): required for a usable student lab workflow.
- `Dependency` (0 or 1): blocks other high-value work.
- `Effort` (1..5): engineering cost/complexity.
- `Score = 2*Importance + 2*Necessity + 2*Dependency - Effort`

Sort by highest `Score` first.

Backlog Template

Copy this block for each candidate feature:

```
Title:
Goal:
Category: [GUI | Firmware | Data | Safety | Analysis | Reporting]

Importance (1-5):
Necessity (1-5):
Dependency (0/1):
Effort (1-5):
Score:

Dependencies:
Acceptance Criteria:
- ...
- ...

Notes:
```

Ideas/Features to Consider

```
Title: CSV Export costumasation
Goal: decide what CSV colums are exported
Category: GUI/Data

Importance (1-5): 5
Necessity (1-5):5
Dependency (0/1):
Effort (1-5): 2
Score: 11

Dependencies:
Acceptance Criteria:
- In GUI when pressing start CSV 1st select what is going in the CSV via
chekboxes
- Generate a function to put in excel to format the data when importing in
Excel - at this point the function for all the data looks like this:
  = Table.TransformColumnTypes(
    #"Promoted Headers",
    {
        {"timestamp_iso", type datetimezone},
        {"elapsed_s", type number},

        {"raw_temp_c", type number},
        {"temp_filtered_c", type number},
        {"temp_smooth_c", type number},

        {"pwm", Int64.Type},

        {"p_term", type number},
        {"i_term", type number},
        {"d_term", type number},
        {"pid_out", type number},
        {"pid_bias", type number},

        {"setpoint_bias_c", type number},
        {"setpoint_c", type number},
        {"effective_setpoint_c", type number},

        {"fan_speed_pct", type number},
        {"fan_pwm_raw", Int64.Type},

        {"mode", type text},
        {"state", type text},

        {"manual_pwm_cmd", type number},
        {"hold_pwm", type number},
        {"enter_progress_pct", type number},
        {"exit_progress_pct", type number},

        {"abs_error_c", type number},

        {"run_state", type text},
```

```
        {"fan_inverted", type number}
    },
    "en-US"
)
```

notes: this is a must have for the lab, it allows students to easily import the data into Excel and to format it for analysis. It also allows us to only export the data that is relevant for the lab and to avoid overwhelming students with too much data. It could be implemented in a way that allows us to easily add new sensors in the future without having to change the CSV export code.

Title: dual file final build
Goal: have single executable or an installer so it is a program for the GUI and a program that has to be uploaded to the microcontroller for the firmware, and have a clear separation between the two in terms of code and functionality
Category: GUI/firmweare/final build

Importance (1-5): 3
Necessity (1-5): 5
Dependency (0/1): 1
Effort (1-5): 4
Score: 13

Dependencies: the GUI and the firmware should be developed in parallel but with a clear separation of concerns, so that the GUI can be developed and tested independently of the firmware, and vice versa. The GUI should be designed to work with a defined communication protocol with the firmware, so that we can easily swap out the firmware or the GUI without having to change the other. The final build should include an installer for the GUI that sets up all the necessary dependencies and configurations, and it should also include a way to upload the firmware to the microcontroller, such as through a USB connection or through an SD card. The final build should also include clear instructions for how to install and use the GUI and how to upload the firmware, so that it is easy for students and instructors to get started with the lab.
Acceptance Criteria:
- the final build should include a single executable or an installer for the GUI, and a separate program for the firmware that can be uploaded to the microcontroller
- the GUI and the firmware should have a clear separation of concerns, with well-defined communication protocols between the two
- the final build should include clear instructions for how to install and use the GUI and how to upload the firmware, so that it is easy for students and instructors to get started with the lab
- ...

Notes: The firmweare should not be nessary becase the end goal is also to have a fully build setup that can be used by students and instructors without having to worry about the technical details of how to upload the firmware, but it is important to have a clear separation between the GUI and the firmware in terms of code and functionality, so that we can easily develop and test them

independently, and so that we can easily swap out one or the other if needed. The final build should be designed with ease of use in mind, so that it is accessible to students with no prior experience with programming or electronics, and it should also be designed to be robust and reliable, so that it can withstand the rigors of a student lab environment. the firmweaer is also given for in the instance if we want to have a setup where the students can upload their own firmware to the microcontroller to test different control algorithms or different sensor configurations, but it is not essential for the core functionality of the lab, and it could be added as an optional feature after we have a stable and usable version of the GUI with the core features. or the firmweare becomes corruptes or other issues arise with the microcontroller, having the firmware available allows us to quickly re-upload it and get the lab back up and running without having to wait for a new microcontroller or a new pre-flashed firmware to arrive.

Title: graph cursor
Goal: have the abilithy to add a cursor in the graph in GUI
Category: GUI

Importance (1-5): 3
Necessity (1-5): 3
Dependency (0/1):
Effort (1-5): 1
Score: 7

Dependencies:
Acceptance Criteria:
- can be toggeled on or off
- tells the values of all the turned on graphs in its on menu
-is easaly moovable (sliding on graph not a slider outside of graph)

notes: this is a nice to have but not a must have, it would make it easier for students to read values at specific times and to compare values at different times, but it is not essential for the core functionality of the lab. It could be added after we have a stable and usable version of the GUI with the core features.

Title: Presets
Goal: have Presets in GUI for diffrent setups/PID setups and diffrent heatsincs and be able to save a setup to a costum named new preset
Category: GUI

Importance (1-5): 2
Necessity (1-5): 4
Dependency (0/1):
Effort (1-5): 2
Score: 8

Dependencies:
Acceptance Criteria:
- be able to save the current setup as a preset with a costum name
- be able to load a preset and have all the settings in the GUI change to the preset
- be able to delete a preset
- be able to edit a preset (load it, change it, save it with the same name or a new name)
- have some default presets for the different heatsinks and for diffrent PID setups (e.g. one with a lot of D, one with no D, one with a lot of P, etc.)

Notes: this is a nice to have but not a must have, it would make it easier for students to get started and to compare different setups, but it is not essential for the core functionality of the lab. It could be added after we have a stable and usable version of the GUI with the core features.

Title: user manual for GUI
Goal: have a user manual for the GUI that explains how to use it and what each feature does
Category: GUI

Importance (1-5): 2
Necessity (1-5): 4
Dependency (0/1): 0
Effort (1-5): 1
Score: 8

Dependencies: /
Acceptance Criteria:
- the manual should be easy to understand and follow for students with no prior experience with PID control or data analysis
- the manual should include screenshots of the GUI and examples of how to use each feature
- the manual should be available in a digital format (e.g. PDF) that can be easily accessed and shared with students
- the manual should be updated as new features are added to the GUI
- the manual should include a troubleshooting section for common issues that students may encounter when using the GUI
- the manual should include a section on how to interpret the data and metrics generated by the GUI, including how to use the cursor tool and how to analyze the graphs.
- the manual should explain how to setup Excel to import the CSV data and how to use the provided Excel function to format the data for analysis. And visualize the data in a scatter graph with the relevant metrics (e.g. rise time, settling time, overshoot, etc.)
- ...

Notes: this is a must have for the lab, it will help students to understand how to use the GUI and to get the most out of it, it will also help to reduce the

number of questions and issues that students may have when using the GUI, and it will make it easier for instructors to teach the lab and to troubleshoot any issues that may arise. It should be developed in parallel with the GUI development, so that it can be updated as new features are added.

Title: tooltips GUI
Goal: have tooltips in the GUI that explain what each feature does when the user hovers over it
Category: GUI

Importance (1-5): 3
Necessity (1-5): 4
Dependency (0/1): 0
Effort (1-5): 1
Score: 8

Dependencies:
Acceptance Criteria:
- when the user hovers over a feature in the GUI, a tooltip should appear that explains what the feature does and how to use it
- the tooltip should be easy to understand and should provide enough information for students to use the feature without having to refer to the user manual
- the tooltip should be available for all features in the GUI, including buttons, sliders, and graphs
- the tooltip should be updated as new features are added to the GUI

Notes: this is a nice to have but not a must have, it would make it easier for students to understand how to use the GUI and to get the most out of it, but it is not essential for the core functionality of the lab. It could be added after we have a stable and usable version of the GUI with the core features, and it could be developed in parallel with the user manual, so that it can be updated as new features are added.

Title: Expert/learning mode
Goal: have an expert mode and a learning mode in the GUI, where the learning mode has more guidance and explanations for students, and the expert mode has more advanced features for instructors or advanced students
Category: GUI

Importance (1-5): 2
Necessity (1-5): 3
Dependency (0/1): 0
Effort (1-5): 3
Score: 8

Dependencies:
Acceptance Criteria:

- the GUI should have a toggle to switch between expert mode and learning mode
- in learning mode, the GUI should provide more guidance and explanations for students, such as tooltips, a user manual, and a simplified interface that focuses on the core features of the lab
- in expert mode all the parameters and features should be available for instructors or advanced students who want to explore more advanced concepts or who want to customize the lab setup
- the GUI should remember the last mode used and should default to that mode when it is opened
- the GUI should allow instructors to lock certain features or parameters in learning mode to prevent students from changing them and to ensure that they are following the intended lab procedure
- the GUI should allow instructors to customize the learning mode interface to focus on specific features or concepts that they want to emphasize in their lab sessions
- the GUI should provide a way for instructors to access and manage the different modes, such as a settings panel or a dedicated mode switcher

Notes: this is a nice to have but not a must have, it would make it easier for students to understand how to use the GUI and to get the most out of it, and it would also allow instructors to customize the lab experience for their students, but it is not essential for the core functionality of the lab. It could be added after we have a stable and usable version of the GUI with the core features, and it could be developed in parallel with the user manual and tooltips, so that it can be updated as new features are added.

Title: PID learning mode or Heat Transfer learning mode
Goal: have a PID learning mode or a Heat Transfer learning mode in the GUI, where students can learn about the concepts of PID control or heat transfer where all the PID settings are fully setup and automatically adjusted to show the effects of changing one parameter at a time, and where the GUI provides explanations and guidance for students to understand the concepts and to see the effects of their changes in real time
Category: GUI

Importance (1-5): 1
Necessity (1-5): 2
Dependency (0/1): 0
Effort (1-5): 4
Score: 8

Dependencies:
Acceptance Criteria:
- the GUI should have a toggle to switch to PID learning mode or Heat Transfer learning mode
- in PID learning mode the GUI is layed out to maximize the understanding of the effects of changing each PID parameter, for example by showing the P, I, D, and output decomposition with hints on how to adjust the parameters to achieve desired effects (e.g. high overshoot: lower Kp or raise D, steady-state offset: increase Ki or bias)

```
- in Heat Transfer learning mode the GUI is layed out to maximize the
understanding of the heat transfer concepts, for example by showing the
temperature distribution across the heatsink, the heat flux, and the effects of
changing the airflow or the ambient temperature on the heat transfer
performance
- in both modes the GUI should provide explanations and guidance for students
to understand the concepts and to see the effects of their changes in real
time, such as tooltips, a user manual, and interactive elements that allow
students to experiment with the parameters and to see the results in the graphs
and metrics
- the GUI should allow students to switch between the learning modes and the
regular mode, so that they can apply what they have learned in the learning
modes to the regular lab experiments
- the GUI should provide a way for instructors to customize the learning modes,
such as by allowing them to choose which parameters to focus on, or by
providing additional explanations or examples for specific concepts that they
want to emphasize in their lab sessions


Notes: this is a nice to have but not a must have, it would provide an
interactive and engaging way for students to learn about PID control and heat
transfer concepts, and it would allow them to see the effects of their changes
in real time, but it is not essential for the core functionality of the lab. It
could be added after we have a stable and usable version of the GUI with the
core features, and it could be developed in parallel with the user manual and
tooltips, so that it can be updated as new features are added.
```

Suggested Priorities (Given Current Project Goal)

P0 (Now)

1. Structured experiment workflow

- Add phase markers/events in graph + CSV.
- Add core auto-metrics (rise time, settling, overshoot, steady-state error).

2. Stable/upgrade-proof data model

- Introduce schema versioning (`schema_version`) and run metadata (`run_id`, `heatsink_id`).
- Keep placeholders for upcoming sensors.

3. Safety + reproducibility baseline

- Ensure `RUN ON/OFF`, sensor-fault behavior, and configuration capture per run.

P1 (Power Sensor Arrival)

1. Add power telemetry

- Log `voltage`, `current`, `power_in`.

2. Equilibrium/dissipation analysis

- Detect steady state and estimate dissipation from power balance.

3. Comparison KPIs

- Per-run summary for heatsink ranking under same conditions.

P2 (Pressure + Airspeed Sensor Arrival)

1. Airflow telemetry

- Log `airspeed` and `delta_p`.

2. Closed-loop airspeed control

- Add a fan control loop to maintain target airspeed.

3. Condition lock for fair comparison

- Test at fixed setpoint + fixed airspeed + recorded pressure drop.

1. Experiment Presets

- Save/load full test setups (SP, mode, gains, fan, thresholds, y-scale).
- One-click presets like `Step Response`, `Disturbance Test`, `Manual Bias Find`.

2. Guided Lab Mode

- Wizard flow: `Connect -> Preheat -> Run -> Disturb -> Analyze -> Save`.
- Lock irrelevant controls per step to reduce mistakes.

3. Auto Test Scripts

- Scripted sequences:
  - Setpoint steps (`30 -> 60 -> 80 C`)
  - Fan disturbance pulse
  - Hold test (steady-state)
- Write phase markers to CSV.

4. Performance Metrics Auto-Compute

- Show `rise time`, `settling time`, `overshoot`, `steady-state error`, `IAE/ISE`.
- Compute metrics per phase and export.

5. Cursor/Probe Tool

- Click graph to read exact values at time `t`.
- Delta mode between two points: `ΔT`, `ΔPWM`, `Δt`, slope.

6. Annotation Markers

- Buttons to add markers (`Fan ON`, `Heatsink Changed`, `Mode Switch`) to CSV/log.
- Draw vertical marker lines with labels on graph.

7. Safety Layer

- Configurable max-temp cutoff, max-PWM duration, sensor-fault lockout.
- Big status indicator: `SAFE / RUNNING / FAULT`.

8. Heatsink Comparison View

- Overlay multiple runs from CSV.
- Normalize time to event (e.g., step start).
- Compare key metrics side by side.

9. Student Report Export

- Generate PDF/HTML report with:
  - Graphs
  - Config used
  - Metrics
  - Student notes
  - Pass/fail checks

10. PID Learning Overlay

- Show `P`, `I`, `D`, and output decomposition with hints:
  - High overshoot: lower `Kp` or raise `D`
  - Steady-state offset: increase `Ki` or bias

11. Access Levels

- `Student mode`: limited controls.
- `Instructor mode`: full tuning, thresholds, scripting.

12. Scoring/Checklist

- Define rubric targets (e.g., overshoot < 5%, settling < 120s).
- Auto-score each run for objective grading.

Suggested Next Two

1. Add phase markers + annotations in CSV/graph.
2. Add automatic rise/settling/overshoot metrics panel.